CLASE 3 · MÓDULO 3 (PARTE 1)

Red neuronal simple: activación y gradiente

Neurona moderna, funciones de activación y descenso por gradiente

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 2
0:10	Del escalón a la neurona moderna
0:25	¿Qué es una derivada?
0:35	Funciones de activación
0:55	Ejemplo a mano
1:00	Práctica: neurona con sigmoide
1:20	Descanso
1:30	Descenso por gradiente
1:55	Ejercicios en papel
2:05	Práctica: descenso por gradiente
2:25	Cierre y tarea

Al terminar la clase podrás…

1

Distinguir

la neurona moderna del perceptrón: activación continua.

2

Calcular

las activaciones lineal, sigmoide, tanh y ReLU.

3

Explicar

qué es una derivada: la pendiente.

4

Aplicar

el descenso por gradiente y elegir η.

5

Programar

una neurona y el descenso en Python.

Repaso de la Clase 2

¿Cómo calcula?

$h = \Sigma\, w_i x_i + b$ y luego el escalón: 1 si $h \geq 0$.

¿Cómo aprende?

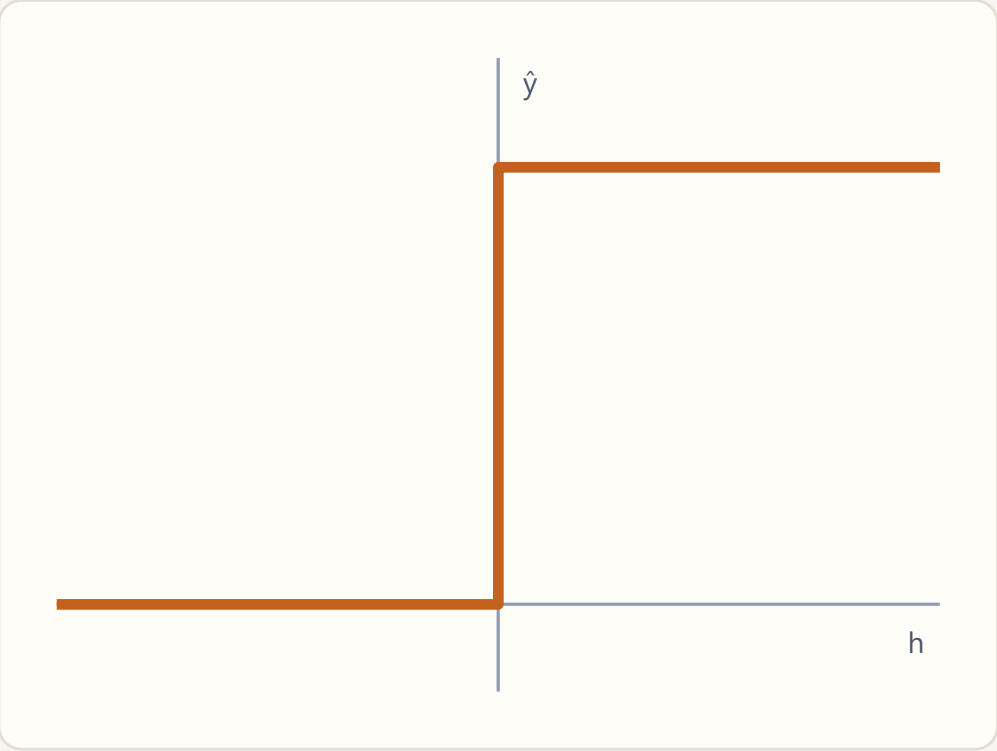
$W \leftarrow W + \eta(y - \hat{y})X$: corrige con el error.

¿Qué no pudo?

XOR: no es linealmente separable.

Hoy: cambiar el escalón por algo suave y aprender a bajar montañas.

¿Qué tiene de malo el escalón?



Escalón: salta de 0 a 1 en h = 0

- **Todo o nada:** h = −0.01 y h = −100 dan lo mismo.
- **Plano:** mover un poco un peso casi nunca cambia la salida.
- **Salto:** en h = 0 no hay pendiente definida.

Buscar llaves con “sí / no” es difícil; con “frío… tibio… caliente” es fácil.

La neurona artificial moderna

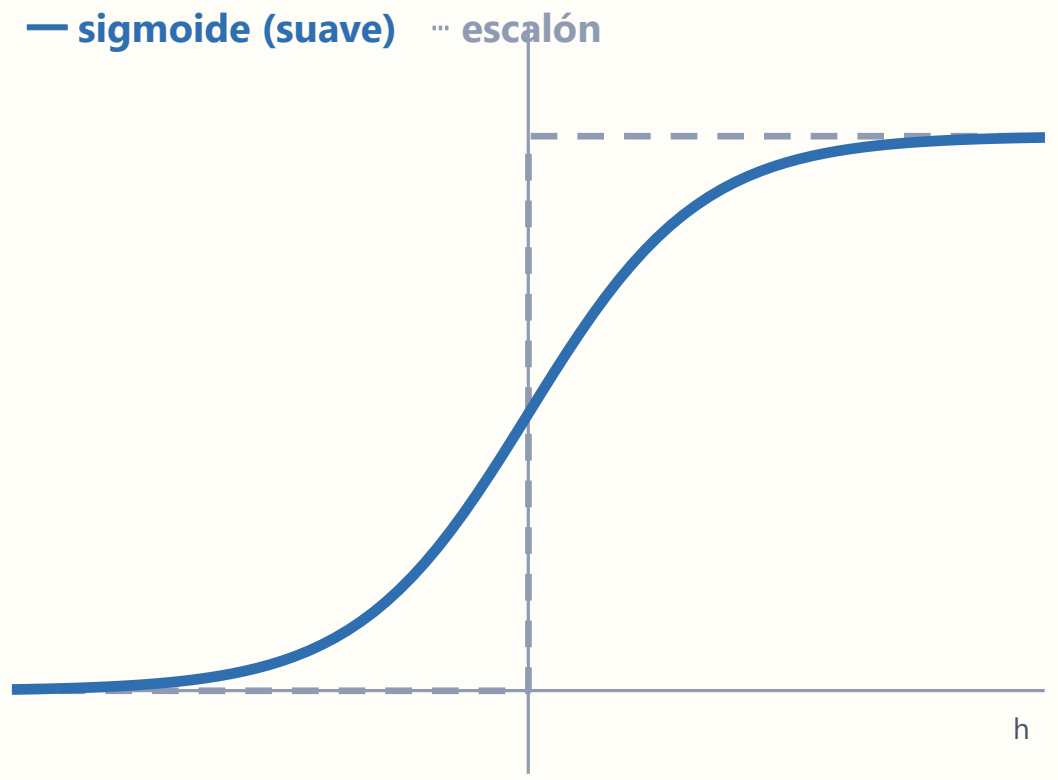
Widrow y Hoff, 1960

ADALINE: ADAptive LInear NEuron.

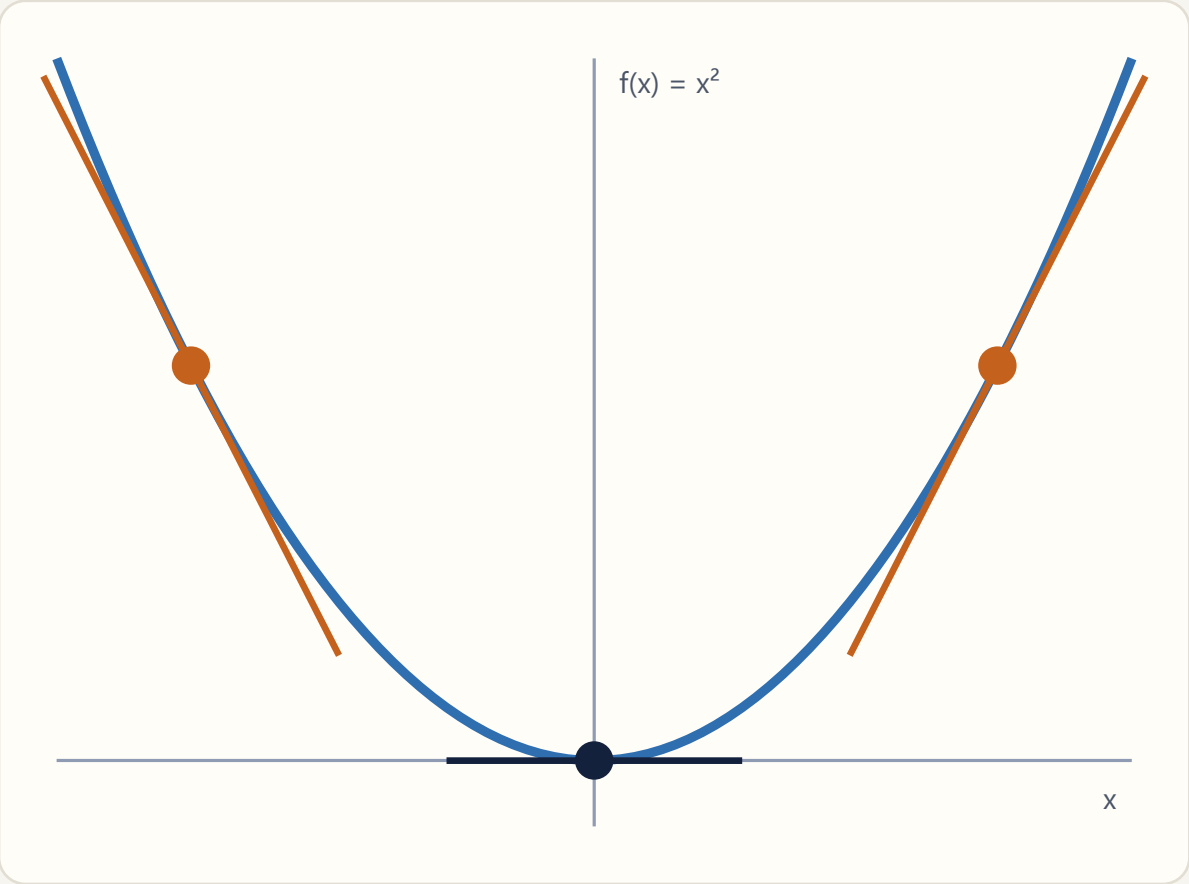
Los mismos dos pasos

$$h = \Sigma\, w_i x_i + b$$

$$\hat{y} = f(h) \quad \text{con } f \textbf{ continua y derivable}$$



¿Qué es una derivada?



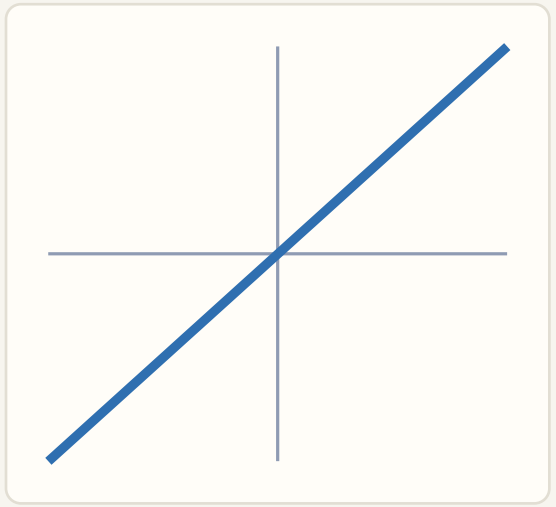
Derivada = pendiente de la curva en un punto: cuánto sube si avanzas un poquito a la derecha.

x	−3	0	1	3
f′(x) = 2x	−6	0	2	6

Signo: hacia dónde sube.

Tamaño: qué tan empinada está.

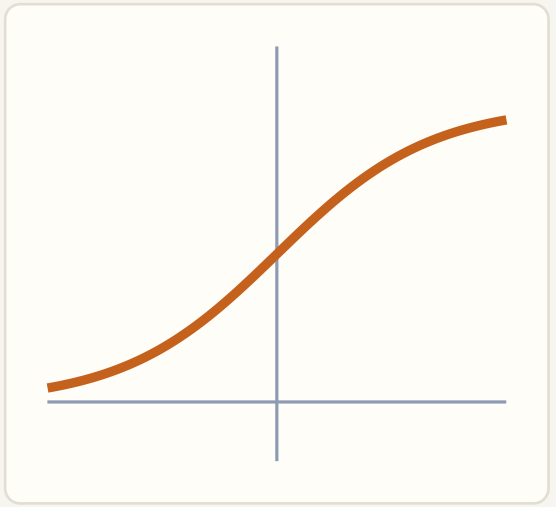
Funciones de activación



Lineal

$f(h) = h$

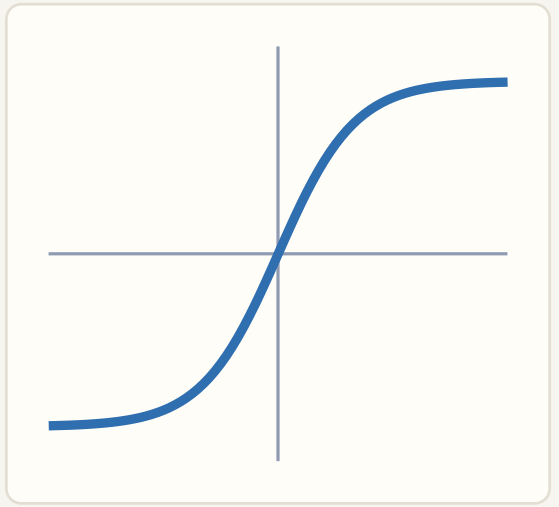
cualquier número



Sigmoide

$\frac{1}{(1 + e^{-h})}$

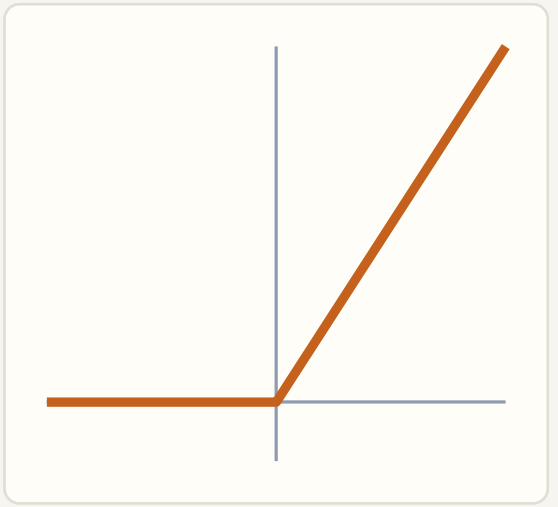
entre 0 y 1



tanh

$\frac{(e^{h} - e^{-h})}{(e^{h} + e^{-h})}$

entre −1 y 1



ReLU

$máx(0, h)$

0 o positivo

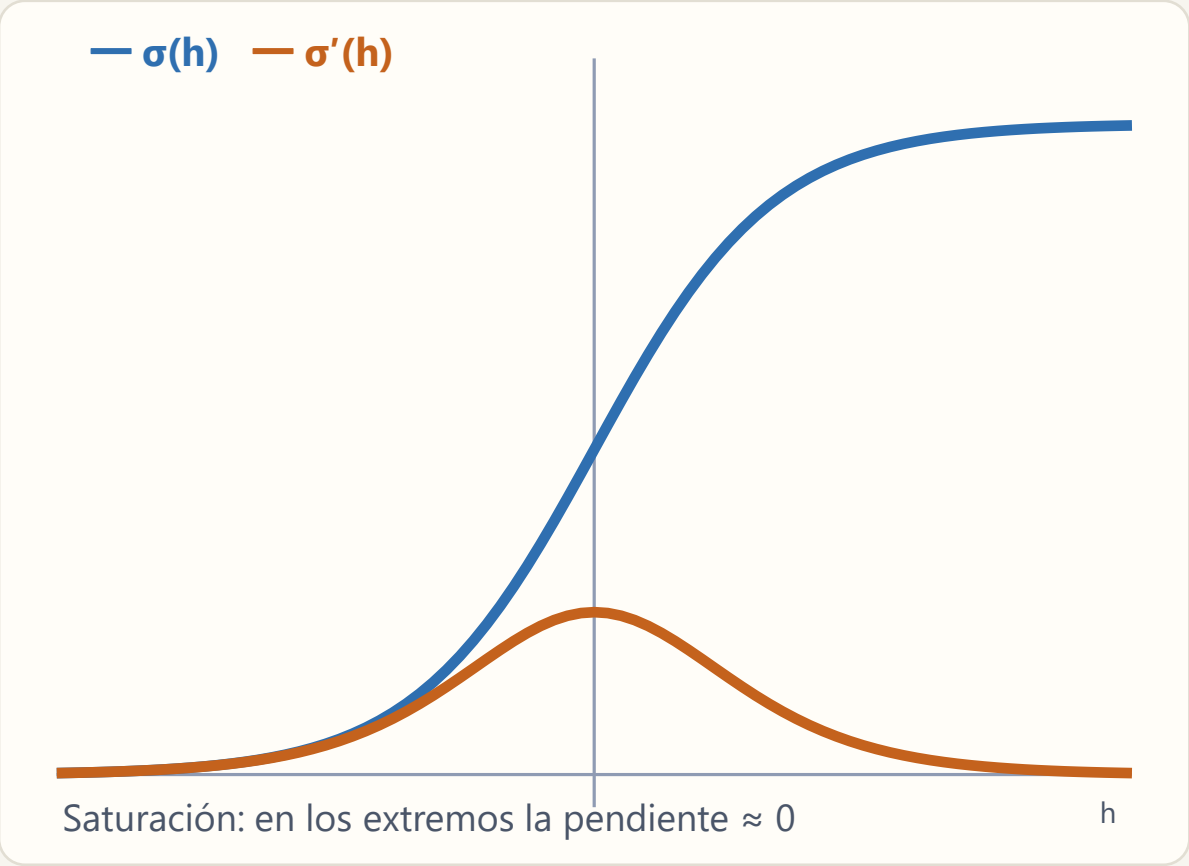
e ≈ 2.718 (como π ≈ 3.14). e^{-h} = “e elevado a menos h”.

Tabla de valores

h	lineal	sigmoide	tanh	ReLU
−3	−3	0.0474	−0.9951	0
−1	−1	0.2689	−0.7616	0
0	0	0.5	0	0
1	1	0.7311	0.7616	1
3	3	0.9526	0.9951	3

La sigmoide nunca llega a 0 ni a 1, y vale exactamente 0.5 cuando h = 0.

La sigmoide en detalle



$$\sigma'(h) = \sigma(h) \cdot (1 - \sigma(h))$$

Máxima en h = 0: σ′(0) = 0.25.

Saturación: con h muy grande o muy negativo aprende lento.

No lineal: si todo fuera lineal, la red entera seguiría siendo una recta.

Ejemplo a mano

`X = [0.7, −0.3]  ·  W = [0.1, 0.8]  ·  b = −0.1`

Paso	Cálculo
1. Combinación lineal	h = 0.07 − 0.24 − 0.1 = −0.27
2. Sigmoide	e^0.27 ≈ 1.3100 → σ(−0.27) = 1 / 2.3100 ≈ 0.4329
3. Con tanh	tanh(−0.27) ≈ −0.2636
Con el escalón	h < 0 → ŷ = 0 (sin más información)

La sigmoide dice algo más rico: “0.43, casi en la frontera”.

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_04_red_neuronal_simple

TODO 1 pt sigmoide(h) con np.exp(-h)

TODO 1 pt Neurona: combinacion_lineal y forward

TODO 1 pt tanh y comparación con la sigmoide

Pistas: np.dot(self.W, X) hace la suma de productos; dentro de la clase usa self.W y self.b.

```
# comprobaciones
Output: 0.4329        # sigmoide
tanh(-0.27) = -0.2636

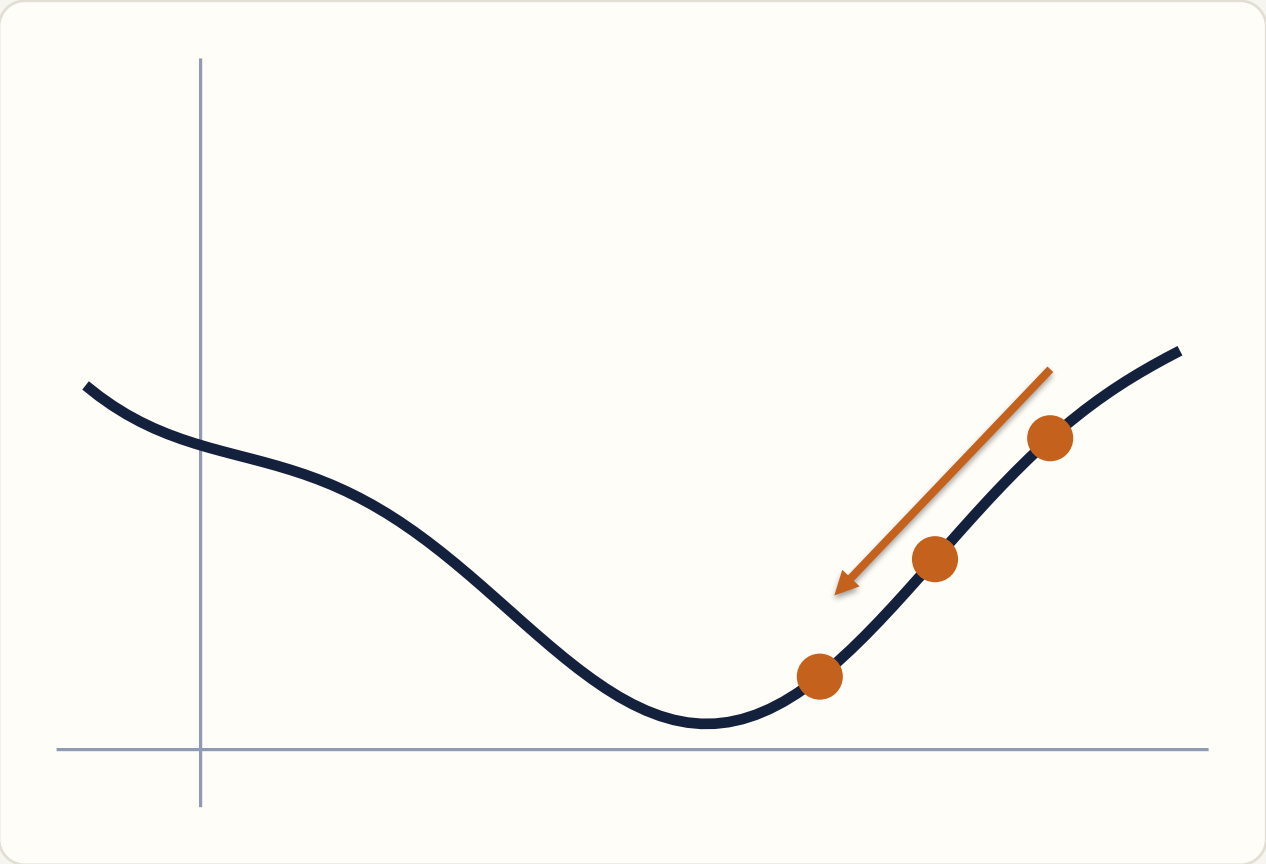
# error típico: np.exp(h)
# sin el signo menos -> 0.5671
```

Sin Python: abre el notebook en Google Colab.

Descanso · 10 min

Después: el descenso por gradiente

Descenso por gradiente



Objetivo: encontrar el x que hace f(x) lo más pequeña posible.

En redes: f es el error y x son los pesos.

De noche, sin linterna: sientes con los pies hacia dónde baja y das un paso.

La regla del descenso

$$x \leftarrow x - \eta \cdot f'(x)$$

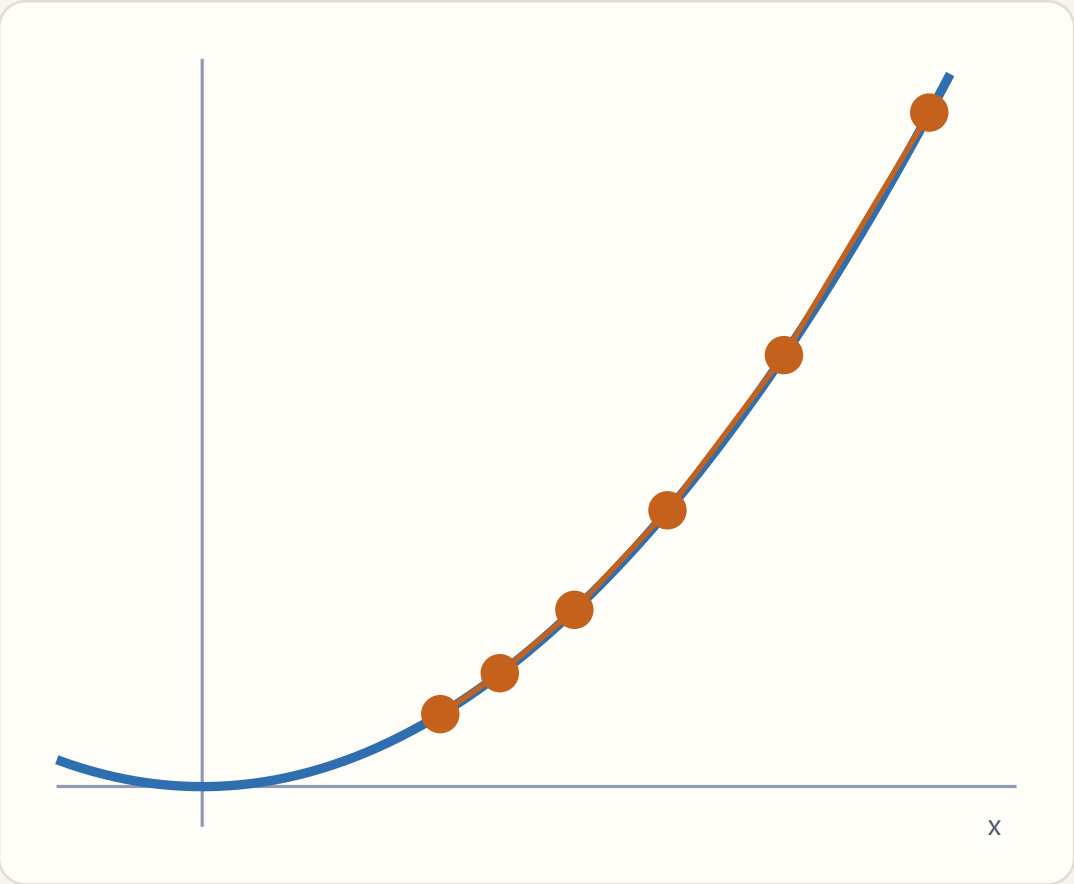
Símbolo	Significa
x	Dónde estás parado
f′(x)	La pendiente: hacia dónde sube
η (eta)	Tasa de aprendizaje: tamaño del paso

¿Por qué el signo menos?

La derivada apunta hacia donde la función sube.

Queremos bajar: vamos en **contra de la pendiente.**

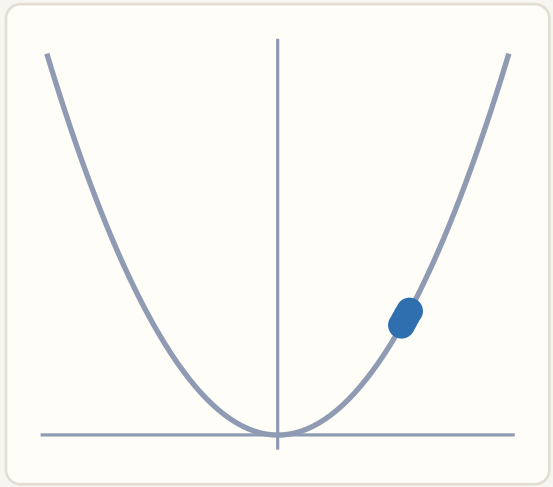
Ejemplo: $f(x) = x^2$, $\eta = 0.1$



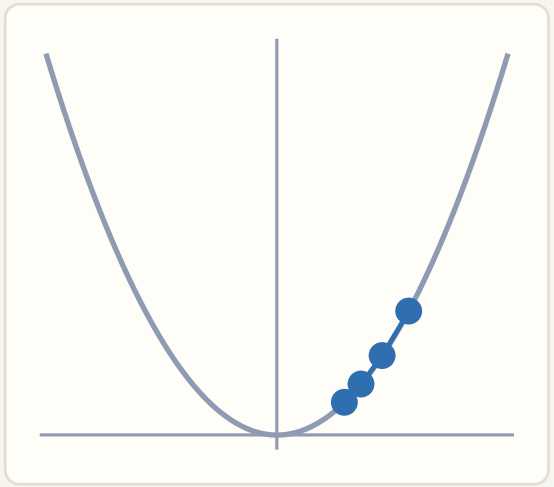
x	f′(x) = 2x	x − 0.1·f′(x)
5	10	4
4	8	3.2
3.2	6.4	2.56
2.56	5.12	2.048
2.048	4.096	1.6384

Los pasos se acortan solos al acercarse al fondo: la pendiente es más pequeña ahí.

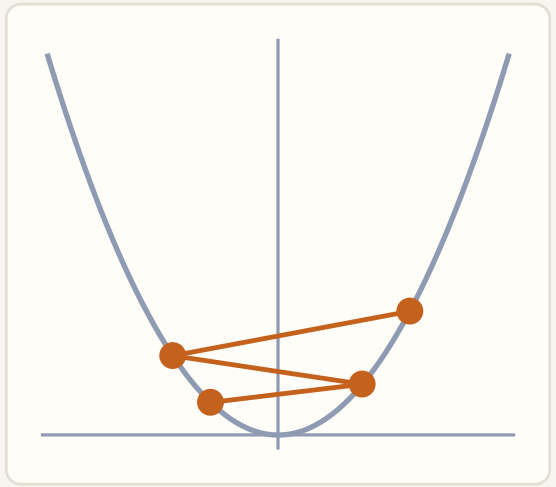
La tasa de aprendizaje η



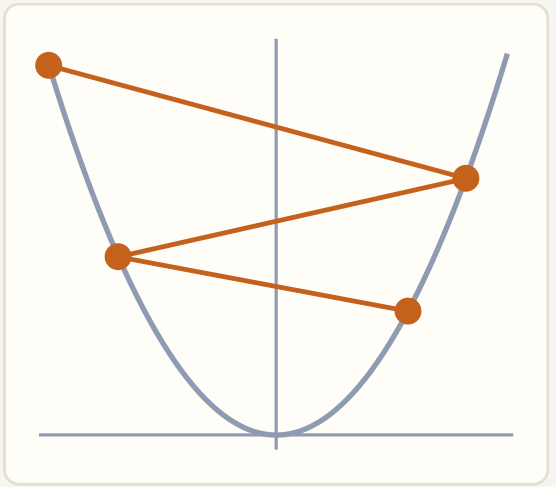
η = 0.01 · muy lento



η = 0.1 · bien



η = 0.9 · oscila



η = 1.1 · diverge

η	0.01	0.1	0.5	0.9	1.1
Desde x = 5, cinco pasos	4.52	1.64	0 (de un salto)	−1.64 (oscila)	−12.44 (diverge)

η es un hiperparámetro: lo elegimos nosotros. Ni muy pequeño ni muy grande.

Ejercicios en papel · 10 min

1

$f(x) = x^2$

$x_0 = 3 \cdot \eta = 0.25$

Calcula dos pasos.

2

$f(x) = 2x + x^2$

$x_0 = 10 \cdot \eta = 0.2$

Pista: $f'(x) = 2 + 2x$

Calcula 6 valores.

3

Sigmoide en $h = 0$

¿Cuánto vale $\sigma(0)$? ¿Y $\sigma'(0)$?

NOTEBOOKS · 15 MIN · 2 PUNTOS · + DOCS/PRACTICA_03.PY

Práctica · 1_05_descenso_por_gradiente_1D

TODO 1 pt gradient_descent_update: la regla del descenso

TODO 1 pt ciclo while hasta error < 1e-3; contar iteraciones

Pistas: la derivada ya existe como df(x). Para detenerse, usa while y un contador.

```
# historial esperado
[5, 4.0, 3.2, 2.56, 2.048, 1.6384]

# iteraciones con 1e-3:
# |x_nuevo - x| -> 32   f(x) -> 23
```

Demo: python docs/practica_03.py --graficar (activaciones y cinco valores de η)

Para llevar

1. Neurona moderna: activación suave y derivable.
2. Derivada = pendiente: dice hacia dónde sube.
3. Descenso: $x \leftarrow x - \eta \cdot f'(x)$, con η ni muy grande ni muy pequeña.

Próxima clase: bajar por el error de la red moviendo los pesos.

Tarea: notebooks 1_04 y 1_05 (5 pts)

Leer: libro, “Descenso por gradiente” (aprendizaje)